TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Trabajo Práctico - Unidad 6

Mantenimiento, respaldo y recuperación

Actividad breve de laboratorio - Semana 10

Proyecto integrador: Food Store | PostgreSQL 16+

Material preparado para trabajo académico y laboratorio. Los ejemplos de comandos se ejecutan primero sobre entornos de prueba y bases descartables.

Propósito

El trabajo integra respaldo, monitoreo, mantenimiento y recuperación sobre la base Food Store. El equipo construye un procedimiento pequeño pero completo: genera una copia, registra métricas, restaura la copia en un entorno descartable y mide el tiempo de recuperación. OpenCode se utiliza para generar o ajustar los scripts bajo una especificación escrita; cada comando debe revisarse antes de ejecutarse.

Condiciones de trabajo

- Motor: PostgreSQL 16 o superior.
- Base principal: foodstore.
- Base de restauración: foodstore_restore_test.
- La base foodstore no se elimina ni se reemplaza durante el simulacro.
- Las contraseñas no se escriben en scripts ni se entregan en el repositorio.
- El respaldo se realiza en formato custom para utilizar pg_restore.
- Las pruebas destructivas sólo se ejecutan sobre entornos descartables.

Parte 1. Especificación y plan de mantenimiento

El equipo crea un archivo SPEC_U6.md de una página como máximo. La especificación debe indicar qué se automatiza, con qué frecuencia se ejecutaría y qué condición confirma que la tarea terminó correctamente.

Tarea mínima	Frecuencia propuesta	Evidencia
Respaldo de foodstore	Diaria	Archivo .dump con fecha y log.
Chequeo de salud	Diaria	Salida de health_check.sql.
Revisión de mantenimiento	Semanal	Estado de tuplas muertas, autovacuum y estadísticas.
Restauración de prueba	Semanal	foodstore_restore_test validada.

Uso de OpenCode
OpenCode puede crear la primera versión de los scripts a partir de SPEC_U6.md y AGENTS.md. La entrega debe demostrar que el código fue revisado y probado. No se evalúa la cantidad de texto generado por la herramienta, sino la corrección del procedimiento final.

Parte 2. Respaldo automatizable

El equipo crea backup_foodstore.ps1 o backup_foodstore.sh. El script debe generar un respaldo custom de foodstore, incluir fecha y hora en el nombre, registrar el resultado en una carpeta logs y finalizar con error si pg_dump falla.

Comando base de referencia

```
pg_dump -h localhost -p 5432 -U postgres -d foodstore -Fc -f backups/foodstore_FECHA_HORA.dump
```

- El script no debe contener una contraseña.
- El equipo debe ejecutar el script manualmente antes de considerarlo apto para programación.
- El equipo debe adjuntar una evidencia breve del archivo generado y del log.

Parte 3. Monitoreo y mantenimiento

El equipo crea health_check.sql con, como mínimo, cuatro controles: tamaño de la base, actividad de conexiones, estado de tablas y consultas activas. Además, crea maintenance.sql con una revisión de autovacuum/ANALYZE y una acción de mantenimiento segura y justificada.

Punto de partida para health_check.sql

```
SELECT pg_size_pretty(pg_database_size('foodstore')) AS tamano;

SELECT numbackends, xact_commit, xact_rollback
FROM pg_stat_database
WHERE datname = 'foodstore';

SELECT relname, n_live_tup, n_dead_tup, last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

SELECT pid, state, now() - query_start AS duracion, query
FROM pg_stat_activity
WHERE datname = 'foodstore' AND state <> 'idle';
```

Criterio
VACUUM FULL no debe utilizarse como mantenimiento periódico del trabajo. Si se incluye un VACUUM o ANALYZE manual, la decisión debe justificarse con la observación previa del estado de las tablas.

Parte 4. Simulacro de desastre contra reloj

El equipo utiliza el último respaldo generado y restaura Food Store en una base nueva llamada foodstore_restore_test. El cronómetro comienza inmediatamente antes de crear la base de prueba y finaliza cuando las validaciones mínimas terminan correctamente.

Restauración de referencia

```
createdb -h localhost -p 5432 -U postgres foodstore_restore_test

pg_restore -h localhost -p 5432 -U postgres -d foodstore_restore_test --no-owner --no-privileges
backups/ULTIMO_RESPALDO.dump
```

Después de restaurar, el equipo ejecuta los siguientes controles sobre foodstore_restore_test:

Validación mínima

```
SELECT 'categoria' AS tabla, COUNT(*) AS filas FROM categoria
UNION ALL SELECT 'cliente', COUNT(*) FROM cliente
UNION ALL SELECT 'producto', COUNT(*) FROM producto
UNION ALL SELECT 'pedido', COUNT(*) FROM pedido
UNION ALL SELECT 'detalle_pedido', COUNT(*) FROM detalle_pedido;

SELECT COUNT(*) AS productos_invalidos
FROM producto
WHERE precio < 0 OR stock < 0;

SELECT COUNT(*) AS detalles_invalidos
FROM detalle_pedido
WHERE cantidad <= 0 OR precio_unitario < 0;
```

- Las cantidades de filas se comparan con las registradas antes del respaldo.
- Los controles de productos y detalles inválidos deben devolver cero.
- El equipo registra el tiempo total observado como RTO del simulacro.
- El equipo calcula la antigüedad del respaldo al momento del simulacro y la utiliza para discutir el RPO observado.

Parte 5. Informe post-incidente

El equipo entrega un informe de una página como máximo con la siguiente información:

- Fecha y hora del respaldo utilizado.
- Tiempo total de restauración y validación.
- RPO y RTO previstos para el laboratorio y valores observados.
- Resultado de las validaciones de tablas y restricciones lógicas.
- Un problema real encontrado durante el procedimiento, si lo hubo.
- Una mejora concreta para reducir riesgo o tiempo en la siguiente ejecución.
- Breve indicación de qué parte fue asistida con OpenCode y cómo se verificó antes de aceptarla.

Entregables

Archivo	Contenido
SPEC_U6.md	Requisitos del plan y reglas de seguridad.
backup_foodstore.ps1 o .sh	Script de respaldo probado.
health_check.sql	Consultas de monitoreo.
maintenance.sql	Revisión y mantenimiento justificado.
restore_test.ps1 o .sh (opcional)	Automatización del ensayo de restauración.
informe_u6.pdf	Evidencias, RPO/RTO, resultado del simulacro y mejora propuesta.

No se entrega
No debe incluirse la contraseña de PostgreSQL. El archivo .dump puede omitirse de la entrega si el aula limita el tamaño de los archivos; en ese caso se incorpora evidencia de su creación, tamaño y restauración exitosa.

Criterios de evaluación

Criterio	Puntaje
Especificación clara y plan de mantenimiento coherente	2,0
Respaldo automatizable, seguro y probado	2,0
Monitoreo y mantenimiento técnicamente correctos	2,0
Restauración, validaciones y medición RPO/RTO	3,0
Informe breve, evidencias y verificación del trabajo asistido con OpenCode	1,0

Puntaje total: 10 puntos.

Condiciones para aprobación técnica

- La restauración debe realizarse en foodstore_restore_test y no sobre foodstore.
- El respaldo debe poder restaurarse de forma efectiva.
- Los scripts no deben exponer contraseñas.
- Las consultas de monitoreo deben ejecutarse en PostgreSQL 16+.
- El informe debe diferenciar RPO de RTO y registrar valores observados.
- La intervención de OpenCode debe quedar subordinada a revisión y prueba humana.

Referencias mínimas

1. **PostgreSQL 16 - Backup and Restore.** https://www.postgresql.org/docs/16/backup.html
2. **PostgreSQL 16 - pg_dump.** https://www.postgresql.org/docs/16/app-pgdump.html
3. **PostgreSQL 16 - pg_restore.** https://www.postgresql.org/docs/16/app-pgrestore.html
4. **PostgreSQL 16 - Monitoring Database Activity.** https://www.postgresql.org/docs/16/monitoring.html
5. **PostgreSQL 16 - Routine Database Maintenance Tasks.** https://www.postgresql.org/docs/16/maintenance.html
6. **OpenCode - Instructions.** https://opencode.ai/docs/instructions/